

Reverse 404 — Walkthrough

Event: BUP CSE Fest 2026 CTF (Online Preliminary, 19 Sep 2026)

Category: Reverse Engineering

Points: n/a (prelims) **Difficulty:** easy-medium

Target / artifact: local ELF binary `reverse404` (x86-64, stripped, PIE)

Solved at: 08:35 BST (UTC+6) **Time to solve:** ~10 minutes

```
bupctf{n3v3r_g0nn4_g1v3_y0u_up_n3v3r_g0nn4_l37_y0u_d0wn}
```

The flag is the **access code the binary accepts** — it is typed in, not printed out. The binary only prints `Access restored.` when the correct 56-character code is supplied.

1. What we were given

The challenge description was a single line — *"Why do I keep getting error 404?"* — and one file, `reverse404` (14 KB). `file` identifies it immediately:

```
$ file reverse404
reverse404: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
for GNU/Linux 3.2.0, stripped
```

So: native x86-64, **stripped** (no function names), PIE, non-static. A pure crackme — the program will ask for something and either accept or reject it.

2. Reconnaissance — what I tried and what came back

2.1 First the cheap rungs (no disassembly)

```
$ strings -a -n 5 reverse404 | grep -iE 'flag|ctf|bup|key|pass|congrat|404'
Reverse 404
404: Access not found.
$ strings -a -n 5 reverse404 | grep -E '\{.*\}'
(no output)
```

No flag in plaintext. But the two strings tell us the shape of the program: it prints `Reverse 404`, asks for something, and prints `404: Access not found.` on failure. The word `Access` hints at the success message being `Access restored.` — which `strings` shows right next to it at offset `0x2035`.

What this told us: there is a `puts("Access restored.")` path that we must reach. The flag is therefore most likely *the input that reaches it*.

2.2 Run it, and find out how it takes input

```
$ chmod +x reverse404 && ./reverse404
Reverse 404
Access code: 404: Access not found.
$ echo test123 | ./reverse404
Reverse 404
Access code: 404: Access not found.      # exit=1
```

What this told us: it reads the access code from **stdin** (not `argv`), and a wrong code exits 1.

2.3 Which libc calls are used?

```
$ rabin2 -i ./reverse404
3  0x000010a0 GLOBAL FUNC  puts
4  0x000010b0 GLOBAL FUNC  ferror
5  0x000010c0 GLOBAL FUNC  __stack_chk_fail
6  0x000010d0 GLOBAL FUNC  fgetc
7  0x000010e0 GLOBAL FUNC  fflush
8  0x000010f0 GLOBAL FUNC  fwrite
```

What this told us: the entire import list is `fgetc` + `puts` / `fwrite` + a stack canary. There is **no** `strcmp` **and no** `memcmp` — so this is *not* Pattern A (direct string compare). It must be **Pattern C: transform the input, then compare against a stored array**. That array will be in `.rodata`. Reading it out and inverting the transform is the whole solve.

2.4 The stored ciphertext

`.rodata` is only `0x98` bytes. The strings end at `0x2045`; after some zero padding there is a 56-byte high-entropy blob starting at `0x2060`:

```
$ r2 -q -c 'px 64 @ 0x2060; q' ./reverse404
0x00002060  b8 1d d1 60 9e 8c e8 51 72 6d 48 ec c4 9c 7e 60
0x00002070  cf db 98 9a c1 4b b5 ec ef a9 c4 a5 c7 13 85 dd
0x00002080  33 50 c3 1b 11 8f 00 9d 86 c0 a5 07 74 59 0e d0
0x00002090  3d 44 bf 88 3d 42 30 2d
```

`.rodata` is `0x2000` – `0x2097`, so the blob is exactly **56 bytes** — and `56` will matter in a moment. This is the `r10` array the check loop loads (`lea r10, [0x2060]`).

3. The check — root cause of the "404"

Disassembly of `main` in the loop region (`r2 -e bin.relocs.apply=true -c 'aaa; s 0x12a2; pd 60'`):

```

0x12a2 lea r10, [0x00002060] ; enc = the 56-byte blob
0x12a9 mov r9d, 0x31 ; 49 -> running addend, += 7
0x12af mov r8d, 0x5d ; 93 -> running XOR key, += 29
0x12b5 mov edi, 9 ; index seed, += 17
0x12ba mov r11d, 0 ; accumulator
0x12c0 mov esi, 0 ; i = 0
0x12c5 mov eax, edi
0x12c7 shr eax, 3
0x12ca mov eax, eax
0x12cc imul rax, rax, 0x24924925 ; magic-number division
0x12d3 shr rax, 0x20
0x12d7 imul eax, eax, 0x38
0x12da mov edx, edi
0x12dc sub edx, eax ; edx = edi % 56
0x12de mov eax, r8d
0x12e1 xor al, byte [rsp + rdx] ; <-- XOR with the INPUT byte at index edx
0x12e4 add eax, r9d
0x12e7 mov ecx, esi
0x12e9 imul rcx, rcx, 0x24924925
0x12f0 shr rcx, 0x20
0x12f4 mov edx, esi
0x12f6 sub edx, ecx
0x12f8 shr edx, 1
0x12fa add edx, ecx
0x12fc shr edx, 2 ; edx = esi / 7
0x12ff lea ebp, [rdx*8]
0x1306 sub ebp, edx ; ebp = 7 * (esi/7)
0x1308 mov ecx, esi
0x130a sub ecx, ebp ; ecx = esi % 7
0x130c add ecx, 1 ; cl = (esi % 7) + 1
0x130f rol al, cl ; rotate the low byte left
0x1311 xor al, byte [r10] ; XOR with enc[i]
0x1314 movzx eax, al
0x1317 or r11d, eax ; accumulate (OR)
0x131a add esi, 1 ; i++
0x131d add edi, 0x11 ; +17
0x1320 add r8d, 0x1d ; +29
0x1324 add r9d, 7 ; +7
0x1328 add r10, 1 ; &enc[i+1]
0x132c cmp esi, 0x38 ; 56 iterations
0x132f jne 0x12c5

0x1331 test r11d, r11d
0x1334 je 0x1360 ; r11 == 0 -> "Access restored."
0x1336 lea rdi, str.404:_Access_not_found. ; else -> 404

```

Two things make this look harder than it is:

- `0x24924925` is not encryption — it is the compiler's **magic constant for division by 7 and by 56** (strength-reduced `idiv`). Replicating those five instructions in Python shows they compute exactly `edi % 56` and `esi / 7`, `esi % 7`.
- The four counters (`edi += 17`, `r8d += 29`, `r9d += 7`, `r10 += 1`) are just an obfuscated way of saying "key[k], rotate-by, and index change every round".

The length gate (immediately before the loop)

```
0x1282 test rbp, rbp
0x1285 je  0x1336                ; empty input -> 404
0x128b lea rax, [rbp - 1]
0x128f cmp byte [rsp + rbp - 1], 0xd    ; last char a '\r' (CRLF input)?
0x1294 cmovne rax, rbp
0x1298 cmp rax, 0x38              ; must be 0x38 = 56
0x129c jne 0x1336                ; wrong length -> 404
```

So the accepted input is **exactly 56 characters** (57 if the line ends with `\r\n`). Combined with the 56-byte `enc` array — one constraint per input byte, no slack.

Root cause, in one sentence

The program stores a fixed 56-byte **ciphertext** in `.rodata`, and for each `i` it takes one input byte (at a shuffled index), XORs it with a rolling key, adds a rolling constant, rotates the byte by `(i % 7) + 1`, XORs it with the ciphertext byte, and ORs every result together. The OR must be zero — i.e. **every round must produce exactly 0** — which uniquely determines all 56 input bytes. There is no hash and no comparison string: the accept condition *is* an invertible per-byte transform, so the "password" can be computed instead of guessed.

4. Exploitation — exact steps

Each round `i` (0 ... 55) performs, in 8-bit arithmetic:

```
idx  = (9 + 17·i) mod 56                # which input byte (a bijection: gcd(17,56)=1)
key   = (0x5D + 29·i) & 0xFF
add   = (0x31 + 7·i) & 0xFF
rot   = (i mod 7) + 1
acc  |= rol8( ((key ^ input[idx]) + add) & 0xFF, rot ) ^ enc[i]
```

The accept condition is `acc == 0`, and since `OR` of non-negative bytes is zero only when **every** byte is zero, the real constraint per round is:

```
rol8( ((key ^ input[idx]) + add) & 0xFF, rot ) ^ enc[i] == 0
```

Reverse it step by step — undo the last operation first, and invert each one:

```

pre  = ror8(enc[i], rot)          # undo rol  (rol<->ror)
t    = (pre - add) & 0xFF         # undo add  (add<->sub)
input[idx] = key ^ t              # undo xor   (xor is its own inverse)

```

That is the whole solver. The script (also in `artifacts/solve.py`) also *re-runs the original algorithm* on the recovered string as a self-check, and asserts that the `idx` mapping really is a bijection (all 56 positions get constrained — otherwise we would have free variables):

```

enc = bytes.fromhex(
    "b81dd1609e8ce851726d48ecc49c7e60"
    "cfdb989ac14bb5ecef9c4a5c71385dd"
    "3350c31b118f009d86c0a50774590ed0"
    "3d44bf883d42302d")
M = 0xFFFFFFFF
rol8 = lambda v, n: ((v << (n & 7)) | (v >> (8 - (n & 7)))) & 0xFF
ror8 = lambda v, n: ((v >> (n & 7)) | (v << (8 - (n & 7)))) & 0xFF

inp = [None] * 56
for i in range(56):
    idx = (9 + 0x11 * i) % 56          # verified against the 0x24924925 magic
    key = (0x5D + 0x1D * i) & 0xFF
    add = (0x31 + 0x07 * i) & 0xFF
    rot = (i % 7) + 1
    inp[idx] = key ^ ((ror8(enc[i], rot) - add) & 0xFF)
flag = bytes(inp)

```

Running it:

```

$ python3 solve.py
[+] modulo-56 magic replicated OK
[+] input idx mapping is a bijection (all 56 positions constrained)
[+] candidate bytes:
b'bupctf{n3v3r_g0nn4_g1v3_y0u_up_n3v3r_g0nn4_l37_y0u_d0wn}'
[+] forward simulation of reconstructed input -> True

```

Sample of the decode (first six rounds, full table reproduced by the script):

i	idx= (9+17i)%56	key (0x5D+29i)	add (0x31+7i)	rot	enc[i]	ror(enc)	pre-add	input[idx]
0	9	0x5d	0x31	1	0xb8	0x5c	0x2b	v
1	26	0x7a	0x38	2	0x1d	0x47	0x0f	u
2	43	0x97	0x3f	3	0xd1	0x3a	0xfb	l
3	4	0xb4	0x46	4	0x60	0x06	0xc0	t
4	21	0xd1	0x4d	5	0x9e	0xf4	0xa7	v
5	38	0xee	0x54	6	0x8c	0x32	0xde	0

Why not just patch the branch? Because the flag is not printed by the program — the flag *is* the password. NOP-ing `jne` /forcing `je 0x1360` only prints the words `Access restored.`, with no `bupctf{...}` anywhere (we confirmed with `strings`: no flag-shaped string exists in the binary). Patching would "win" nothing; the transform had to be inverted. That is what makes this a crackme rather than a patch-me.

5. Capturing the flag

Feed the recovered 56-byte string back to the original binary:

```
$ FLAG=$(cat flag_candidate.txt)
$ printf '%s\n' "$FLAG" | ./reverse404
Reverse 404
Access code: Access restored.      # exit=0
```

Negative control — one extra byte makes it fail again:

```
$ printf '%s\n' "${FLAG}x" | ./reverse404
Reverse 404
Access code: 404: Access not found. # exit=1
```

Both CRLF and LF inputs are accepted (`cmp byte [rsp+rbp-1], 0xd` handles the trailing `\r`), which is why the binary works when pasted from a Windows terminal too.

6. Why this worked — the concept

The check is a **bijective per-byte transform**, not a hash and not a string comparison. Because `OR`-accumulation demands that every round be zero, and because the index sequence `9 + 17·i mod 56` visits all 56 positions exactly once, the ciphertext pins down each input byte independently. Each round is one `xor`, one `add`, and one `rol` — all trivially invertible in 8-bit arithmetic — so the "password" is not a secret at all: it is arithmetic that the binary *must* be able to invert itself to accept any input. The obfuscation (magic-number division, rolling counters) is there to make the loop look scary, and none of it adds cryptographic strength.

7. How to recognise this next time

- `rabin2 -i` shows `fgetc / fread` but **no** `strcmp / memcmp` → it is a transform-and-compare crackme, not a direct string compare. Go find the ciphertext in `.rodata` and invert.
- A blob of "random" bytes in `.rodata` whose **length equals a** `cmp reg, <len> constant` right before the check loop → that pair (ciphertext length, required input length) is the whole game.
- `0x24924925`, `0xAAAAAAB`, `0xCCCCCCD`, `0x51EB851F`, `imul + shr 0x20` → compiler magic for **integer division**. Read them as `/3`, `/7`, `/56` and the loop becomes readable.
- `or r11d, eax` accumulating instead of `cmp`-per-byte → the accept condition is "every byte of the transform is zero", which is exactly an invertible constraint.
- Before fighting the loop: check whether the success **message** is the flag (`strings | grep '{'`). Here it is not — so patching is a dead end and the transform must be inverted. Always settle this question first; it decides patch vs. solve.

8. Appendix — scripts, payloads, artifacts

- `artifacts/solve.py` — full solver: replicates the magic-number modulo, builds the bijection map, inverts `xor → add → rol`, re-simulates the original loop as a self-check, and writes `flag_candidate.txt`.
- The challenge binary is kept out of this folder on purpose (it belongs to the event); re-download it or ask for a copy of `artifacts/` if you want to re-run the verification.

Verification performed, in order: 1. `python3 solve.py` → candidate + `forward simulation -> True`. 2. `printf '%s\n' "$FLAG" | ./reverse404` → `Access restored.`, exit 0. 3. `printf '%s\r\n' "$FLAG" | ./reverse404` → `Access restored.`, exit 0. 4. `printf '%s\n' "${FLAG}x" | ./reverse404` → `404: Access not found.`, exit 1.

Briefing crib sheet

The binary reads **56 characters** from stdin, then runs a 56-round loop where round `i` takes `input[(9+17i) mod 56]`, XORs it with key `(0x5D+29i)`, adds `(0x31+7i)`, rotates the byte left by `(i mod 7)+1`, XORs with `enc[i]` from the 56-byte `.rodata` blob at `0x2060`, and ORs all results together — accept only if the OR is zero, so every round must be exactly zero. `0x24924925` is just the compiler's magic constant for division by 56/7, not crypto. I inverted it per round: `pre = rol8(enc[i], rot)`, then `pre - add`, then XOR `key`, giving the byte at index `(9+17i) mod 56`; since `gcd(17,56)=1` the index map is a bijection, so all 56 bytes are pinned down and the code is the flag `bupctf{n3v3r_g0nn4_g1v3_y0u_up_n3v3r_g0nn4_l37_y0u_d0wn}`. Patching the branch would only print `Access restored.` — the flag is the password, not the message. Bug class: **invertible obfuscation mistaken for a secret** — the program must contain everything needed to accept the right input, so any input check that is not a hash can be reversed.